IMPLEMENTATION PLAN

Initial-state spreadsheet → well volumes + labware-mapping grounding

nl2protocol · website-voice-rewrite branch

Context

Today the pipeline has no way to learn the **actual starting volume** of a well.

When an instruction says "well A1 contains lysis buffer" without a volume, `WellCapacitySuggester` (`gap_resolution/suggesters.py:218`) guesses the labware's *capacity* ("assume full") at confidence 0.7 and forces the user to confirm it in a modal. That guess is the only number available for the downstream arithmetic checks (overdispense / overflow), and it is usually wrong — the real starting volume is a setup fact only the experimenter knows. Real labs record this by hand; the robot has no sensor for it.

This change lets the user upload a spreadsheet that records, per well, what it holds and how much. The spreadsheet is an **independent artifact**: it is parsed on its own terms into an initial-state object, never fused with the instruction. A separate reconciliation step then uses that object to fill volumes, to ground which labware nickname maps to which config label, and to validate that every container the instruction draws from is actually loaded.

A `csv_path` parameter already exists on `run_pipeline` (`pipeline.py:1462`), wired from the CLI `-d/--data` flag, but is never read in the function body — dead code we will repurpose or remove.

Decisions (locked with user)

- **Format: Excel only** (`.xlsx`), parsed server-side with **openpyxl** (new dependency).
- **Sheet is keyed on config labels**, not user nicknames. The deck-setup person knows the real config labels; this is what makes the mapping crosscheck possible.
- **Sheet is a standalone artifact.** Parser: xlsx bytes → `InitialState` object. No instruction involved in parsing.
- **Browser-only input.** No CLI. File bytes travel base64 in the `/start` body, decoded and parsed server-side.
- **Phased**, held as one feature (no early merge): Phase 1 parse + fill volumes; Phase 2 mapping hint + source validation. Both ship together end-to-end.

The sheet template (the "writing rule")

One row per loaded well:

labware_label	well	substance	volume_ul
source_plate	A1	lysis buffer	250
reagent_reservoir	A1	ethanol	12000

- `labware_label` must match a key in the lab config's `labware` map.
- `well` is a standard coordinate (A1…H12 etc.).
- `substance` is free text (matched loosely against the instruction's wording).
- `volume_ul` is a number, in microliters.

Artifact model

New dataclass `InitialState` (new file `nl2protocol/stage_1_pre_extraction/initial_state.py`):

- `rows` — list of `(labware_label, well, substance, volume_ul)`.
- `by_labware()` — groups rows per config label (the well-set used for the mapping crosscheck).
- `lookup(labware_label, well)` — used to fill volumes after mapping resolves.
- `unmatched` accounting — populated during reconciliation so unused rows can be surfaced.

Keep this object separate from `WellContents` (`models/spec.py:378`), which is the instruction-derived shape (its `.labware` is *user wording*). The reconciliation step is what writes sheet data into `spec.initial_contents`.

Implementation

PHASE 1 — parse + fill volumes

1 · Add dependency

Add `openpyxl` to `[project] dependencies` in `pyproject.toml`.

2 · Parser

`parse_initial_state(xlsx_bytes, config) → InitialState` in the new `initial_state.py`:

- Read the first worksheet with openpyxl (`read_only=True`).
- Validate headers against the template; validate each `labware_label` exists in `config["labware"]`; coerce `volume_ul` to float; collect row-level errors into a structured result (do not raise on the first bad row — report all).
- Pure function, no I/O beyond the in-memory workbook. Fully unit-testable.

3 · Plumb the file in (browser → pipeline)

- `server/app.py` `/start` handler (~721–805): accept an optional `initial_state_b64` field; base64-decode to bytes; call `parse_initial_state(...)`; thread the `InitialState` into `_run_pipeline` (app.py:1103) → `ProtocolAgent.run_pipeline`.
- Browser page (the same form that posts instruction + config): add a file input for the `.xlsx`, read it as base64, include `initial_state_b64` in the `/start` body.
- `run_pipeline` (`pipeline.py:1462`): replace the dead `csv_path` param with `initial_state: Optional[InitialState] = None`. Remove the CLI `-d/--data` wiring (`for_cli/cli.py:685, 91–94`) or leave the flag inert — out of scope and unused.

4 · Fill volumes (reconciliation, volume half)

After labware assignments are applied (`_apply_labware_assignments`, ~pipeline.py:1838) and before the IC confirmation modal `_confirm_initial_contents_via_handler` (`pipeline.py:628`): for each `spec.initial_contents` entry whose resolved config label + well hits `initial_state.lookup(...)`, set `volume_ul` from the sheet and stamp `volume_ul_provenance` with a new `provenance_source` value (e.g. `"initial_state_sheet"`; extend the `Literal` on `Suggestion` / `Provenance`, `gap_resolution/types.py:127`).

- Wells covered by the sheet now have a non-null volume → the `InitialContentsVolumeDetector` creates no gap and `WellCapacitySuggester` never fires for them.
- Wells NOT in the sheet still flow into the IC modal with the capacity default (0.7) exactly as today. Pre-filled sheet values appear in the modal as accepted values the user can still edit.

PHASE 2 — mapping hint + source validation

5 · Mapping crosscheck → resolver hint

Before labware resolution (the `_EarlyLabwareResolver(...).suggest(spec)` call, ~pipeline.py:1604): build a hint map by matching each nickname's well-set + substances (from `_collect_descriptions_with_wells()`, `resolver.py:497`) against each config label's well-set in `initial_state.by_labware()`. A nickname whose well/substance set best matches exactly one config label yields `nickname → label`.

- Inject as an optional `external_hints` on the resolver, consumed in `LabwareMatcher._resolve_one` (`resolver.py:569–605`) **after** `_type_filter`, via a new `_apply_external_hints(description, wells, survivors)` that narrows survivors to the hinted label when present.
- Tie-break order on conflict: **sheet crosscheck > shape/category > substance**. On genuine conflict between a hint and the surviving candidates, do **not** silently override — keep the candidates and let the existing llm/unresolvable branch surface it.

6 · Source-is-loaded validation

After reconciliation, cross the instruction's draw-from set (transfer step sources) against known initial contents (sheet + inline). Surface two mismatches as warnings (reuse the existing warning / `StageEvent` channel, not a new mechanism):

- a step draws from a container/well with no initial contents → **"source not loaded"**;
- a sheet row matches no source or destination in the protocol → **"row unused"**.

Do **not** drop unused rows silently.

⚠️ **Adjacent cleanup (recommended, same blast radius)**

There are **two divergent well-capacity tables** for one fact: `_LABWARE_CAPACITY_HINTS` / `_capacity_for_labware` (`suggesters.py:197–215`) and `_capacity_from_load_name` (`validation/constraints.py:~1170`). They can disagree for the same plate — e.g. `nest_96_wellplate_200ul` → the suggester proposes 300 µL while the overflow check believes capacity is 200 µL, so the system flags its own default as overflowing. Consolidate to one shared capacity function both call. Small and optional, but it removes a real self-contradiction in the same volume logic this feature exercises.

Critical files

`nl2protocol/stage_1_pre_extraction/initial_state.py`
NEW — model + parser.

`nl2protocol/pipeline.py`
`run_pipeline` param (1462); reconciliation calls around labware apply (~1838) and IC modal (628); resolver hint build (~1604).

`nl2protocol/extraction/resolver.py`
: `_resolve_one` (569) + new `_apply_external_hints`; reuse `_collect_descriptions_with_wells` (497).

`nl2protocol/server/app.py`
: `/start` handler (721–805), `_run_pipeline` (1103, 1183, 1204) to accept and thread the file.

`browser static asset / template posting to /start`
: Add file input + base64 encode.

`nl2protocol/models/spec.py · nl2protocol/gap_resolution/types.py`
: Add the `initial_state_sheet` provenance source.

`pyproject.toml`
: Add `openpyxl`.

`nl2protocol/validation/constraints.py · suggesters.py`
: Optional capacity consolidation.

Verification

- **Unit (parser):** valid sheet → correct `InitialState`; bad label, missing column, non-numeric volume → reported (not raised); multiple bad rows all reported.
- **Unit (mapping crosscheck):** nickname well/substance set matching exactly one config label → hint produced; ambiguous set → no hint; conflicting hint vs candidates → no silent override.
- **Integration (run_pipeline):** instruction with two unstated source volumes + a sheet covering one → covered well takes the sheet value with `initial_state_sheet` provenance and creates no IC gap; uncovered well still reaches the IC modal with the capacity default. A sheet that names a labware → the resolver picks that config label.
- **Validation:** instruction drawing from an unloaded well → "source not loaded" warning; sheet row used by nothing → "row unused" warning.
- **Browser E2E:** upload a real `.xlsx` alongside an instruction + config in the live UI, run it, and watch the reported volumes / step columns reflect the sheet values — not a code-level check only.

Plan file: `docs/plans/initial-state-spreadsheet.html` · also at `~/.claude/plans/jaunty-cooking-sloth.md`